EAST WEST UNIVERSITY
excellence is education
Department of Computer Science & Engineering

Software Testing & Quality Assurance - Homework

Course Title : Software Testing & Quality Assurance

Course Code : CSE430

Section : 01

Topic: Pytest Unit Testing

Submitted To:

Shartaz Sajid Nahid

Lecturer

Department of Computer Science & Engineering

East West University.

Submitted By:

Mashiur Rahaman Mollah Niran

ID : 2022-3-60-183

CSE430, Secion 1

Date of submission : 1st July, 2026

Table of Content

Homework 1 - Password Strength Checker

i) Code Snippets

password_checker.p password_checker_test.p

```python
import pytest

from password_checker import PasswordChecker

@pytest.fixture(scope="module")  12+ usages
def setup():
    print("i am setup before test")
    checker = PasswordChecker()
    yield checker
    print("i am done after test")

@pytest.mark.parametrize(
    "password, expected",
    [
        ( password "Mashiur183",  expected True),
        ( password "Niran18",  expected False),
        ( password "mashiur183",  expected False),
        ( password "RahamanMollah",  expected False),
        ( password "",  expected False),
    ]
)
```

password_checker.p password_checker_test.p

```python
class PasswordChecker:  2+ usages
    def is_strong(self, password :{__len__, __iter__} ):
        if not password:
            return False

        if len(password) < 8:
            return False

        has_digit = False
        has_upper = False

        for ch in password:
            if ch.isdigit():
                has_digit = True
            if ch.isupper():
                has_upper = True

        return has_digit and has_upper
```

```python
def test_password_strength(
        setup : PasswordChecker , password : str , expected : bool ):
    assert setup.is_strong(password) == expected

def test_strong_password(setup : PasswordChecker ):
    assert setup.is_strong("MollahNiran183") is True

def test_too_short(setup : PasswordChecker ):
    assert setup.is_strong("M183") is False

def test_no_digit(setup : PasswordChecker ):
    assert setup.is_strong("MashiurRahaman") is False

def test_no_uppercase(setup : PasswordChecker ):
    assert setup.is_strong("niranmollah183") is False
```

ii) Screenshot of test runs

✓ 9 tests passed 9 tests total, 0 ms

```
============================= test session starts ==============================
collecting ... collected 9 items

password_checker_test.py::test_password_strength[Mashiur183-True] i am setup before test
PASSED [ 11%]
password_checker_test.py::test_password_strength[Niran18-False] PASSED   [ 22%]
password_checker_test.py::test_password_strength[mashiur183-False] PASSED [ 33%]
password_checker_test.py::test_password_strength[RahamanMollah-False] PASSED [ 44%]
password_checker_test.py::test_password_strength[-False] PASSED          [ 55%]
password_checker_test.py::test_strong_password PASSED                    [ 66%]
password_checker_test.py::test_too_short PASSED                          [ 77%]
password_checker_test.py::test_no_digit PASSED                           [ 88%]
password_checker_test.py::test_no_uppercase PASSED                       [100%]i am done after test


============================== 9 passed in 0.05s ===============================
```

iii) Test-case table

TC No	Method	Input	Expected Output	Result
TC1	is_strong	"Mashiur183"	True	Pass
TC2	is_strong	"Niran18"	False	Pass
TC3	is_strong	"mashiur183"	False	Pass
TC4	is_strong	"RahamanMollah"	False	Pass
TC5	is_strong	""	False	Pass
TC6	is_strong	"MollahNiran183"	True	Pass
TC7	is_strong	"M183"	False	Pass
TC8	is_strong	"MashiurRahaman"	False	Pass
TC9	is_strong	"niranmollah183"	False	Pass

Homework 2 - Movie Ticket Booking

i) Code Snippets

password_checker.p | password_checker_test.p | movie_ticket.py

```python
class Cinema:  9+ usages
    def __init__(self, total_seats):
        self.total_seats = total_seats
        self.booked = 0

    def book_seats(self, n :{__gt__}):  11+ usages
        if n > self.available_seats():
            raise ValueError("Not enough seats available.")
        self.booked += n

    def available_seats(self):  8+ usages
        return self.total_seats - self.booked

    def is_sold_out(self):  5+ usages
        if self.available_seats() == 0:
            return True
        return False
```

movie_ticket_test.py | movie_ticket.py | password_checker.p

```python
import pytest
from movie_ticket import Cinema

@pytest.fixture(scope="module")  3+ usages
def setup():
    print("i am setup before test")
    cinema = Cinema(10)
    yield cinema
    print("i am done after test")

def test_normal_booking(setup :Cinema):
    setup.book_seats(3)
    assert setup.available_seats() == 7

def test_exact_sellout():
    cinema = Cinema(5)
    cinema.book_seats(5)
    assert cinema.available_seats() == 0
    assert cinema.is_sold_out() is True

def test_overbooking():
    cinema = Cinema(4)
    with pytest.raises(ValueError) as err:
        cinema.book_seats(6)
    assert str(err.value) == "Not enough seats available."
```

movie_ticket_test.py × movie_ticket.py password_checker.p pas

```
26
27  def test_booking_zero_seats():
28      cinema = Cinema(8)
29      cinema.book_seats(0)
30      assert cinema.available_seats() == 8
31      assert cinema.is_sold_out() is False
32
33  def test_multiple_booking():
34      cinema = Cinema(12)
35      cinema.book_seats(2)
36      cinema.book_seats(4)
37      assert cinema.available_seats() == 6
38      assert cinema.is_sold_out() is False
39
40  def test_book_one_seat():
41      cinema = Cinema(7)
42      cinema.book_seats(1)
43      assert cinema.available_seats() == 6
44      assert cinema.is_sold_out() is False
45
46  def test_book_remaining_seats():
47      cinema = Cinema(6)
48      cinema.book_seats(2)
49      cinema.book_seats(4)
50      assert cinema.available_seats() == 0
51      assert cinema.is_sold_out() is True
52
53  def test_overbooking_after_partial_booking():
54      cinema = Cinema(5)
55      cinema.book_seats(3)
56      with pytest.raises(ValueError) as err: cinema.book_seats(3)
57      assert str(err.value) == "Not enough seats available."
```

ii) Screenshot of test runs

✓ 8 tests passed 8 tests total, 2 ms

```
============================= test session starts ==============================
collecting ... collected 8 items

movie_ticket_test.py::test_normal_booking i am setup before test
PASSED                          [ 12%]
movie_ticket_test.py::test_exact_sellout PASSED                          [ 25%]
movie_ticket_test.py::test_overbooking PASSED                            [ 37%]
movie_ticket_test.py::test_booking_zero_seats PASSED                     [ 50%]
movie_ticket_test.py::test_multiple_booking PASSED                       [ 62%]
movie_ticket_test.py::test_book_one_seat PASSED                          [ 75%]
movie_ticket_test.py::test_book_remaining_seats PASSED                   [ 87%]
movie_ticket_test.py::test_overbooking_after_partial_booking PASSED      [100%]i am done after test


============================== 8 passed in 0.06s ===============================
```

iii) Test-case table

TC No	Method	Input	Expected Output	Result
TC1	book_seats()	3 from 10 seats	7 available	Pass
TC2	book_seats()	5 from 5 seats	0 available, sold out=True	Pass
TC3	book_seats()	6 from 4 seats	ValueError	Pass
TC4	book_seats()	0 from 8 seats	8 available, sold out=False	Pass
TC5	book_seats()	2 then 4 from 12 seats	6 available	Pass
TC6	book_seats()	1 from 7 seats	6 available	Pass
TC7	book_seats()	2 then 4 from 6 seats	0 available, sold out=True	Pass
TC8	book_seats()	3 then 3 from 5 seats	ValueError on second booking	Pass

Homework 3 - Inventory Restock Checker

i) Code Snippets

inventory_restock_tracker.py

```python
class InventoryItem:  6+ usages
    def __init__(self, quantity, reorder_level):
        self.quantity = quantity
        self.reorder_level = reorder_level

    def restock(self, amount):  2+ usages
        self.quantity += amount

    def sell(self, amount :{__gt__}):  3+ usages
        if amount > self.quantity:
            raise ValueError("Not enough stock available.")

        self.quantity -= amount

    def needs_reorder(self):  2+ usages
        if self.quantity <= self.reorder_level:
            return True
        return False
```

inventory_restock_tracker_test.py

```python
import pytest

from inventory_restock_tracker import InventoryItem

@pytest.fixture(scope="module")  6+ usages
def setup():
    print("i am setup before test")
    item = InventoryItem( quantity: 50,  reorder_level: 10)
    yield item
    print("i am done after test")

def test_restock_item(setup :InventoryItem ):
    setup.restock(20)
    assert setup.quantity == 70

def test_sell_item(setup :InventoryItem ):
    setup.sell(15)
    assert setup.quantity == 55
```

inventory_restock_tracker.py | inventory_restock_tracker_test.py | movie_ticket_test.py | movie_ticket.py

```
17      setup.sell(15)
18      assert setup.quantity == 55
19
20  def test_sell_more_than_available():
21      item = InventoryItem( quantity: 20,  reorder_level: 5)
22      with pytest.raises(ValueError) as err:
23          item.sell(25)
24      assert str(err.value) == "Not enough stock available."
25
26  def test_needs_reorder_true():
27      item = InventoryItem( quantity: 8,  reorder_level: 10)
28      assert item.needs_reorder() is True
29
30  def test_needs_reorder_false():
31      item = InventoryItem( quantity: 25,  reorder_level: 10)
32      assert item.needs_reorder() is False
33
34  @pytest.mark.parametrize(
35      "start_qty, reorder_level, restock_amt, sell_amt, expected_qty",
36      [
37          ( start_qty 30,  reorder_level 5,  restock_amt 10,  sell_amt 15,  expected_qty 25),
38          ( start_qty 20,  reorder_level 5,  restock_amt 5,  sell_amt 10,  expected_qty 15),
39          ( start_qty 15,  reorder_level 5,  restock_amt 20,  sell_amt 25,  expected_qty 10),
40          ( start_qty 40,  reorder_level 10,  restock_amt 0,  sell_amt 20,  expected_qty 20),
41      ]
42  )
43  def test_restock_sell_combinations(
44          start_qty : int , reorder_level : int , restock_amt : int , sell_amt : int , expected_qty : int ):
45      item = InventoryItem(start_qty, reorder_level)
46      item.restock(restock_amt)
47      item.sell(sell_amt)
48      assert item.quantity == expected_qty
```

ii) Screenshot of test runs

✓ 9 tests passed 9 tests total, 0 ms

```
============================= test session starts ==============================
collecting ... collected 9 items

inventory_restock_tracker_test.py::test_restock_item i am setup before test
PASSED              [ 11%]
inventory_restock_tracker_test.py::test_sell_item PASSED                 [ 22%]
inventory_restock_tracker_test.py::test_sell_more_than_available PASSED  [ 33%]
inventory_restock_tracker_test.py::test_needs_reorder_true PASSED        [ 44%]
inventory_restock_tracker_test.py::test_needs_reorder_false PASSED       [ 55%]
inventory_restock_tracker_test.py::test_restock_sell_combinations[30-5-10-15-25] PASSED [ 66%]
inventory_restock_tracker_test.py::test_restock_sell_combinations[20-5-5-10-15] PASSED [ 77%]
inventory_restock_tracker_test.py::test_restock_sell_combinations[15-5-20-25-10] PASSED [ 88%]
inventory_restock_tracker_test.py::test_restock_sell_combinations[40-10-0-20-20] PASSED [100%]i am done after test


============================== 9 passed in 0.03s ===============================
```

iii) Test-case table

TC No	Method	Input	Expected Output	Result
TC1	restock()	quantity=50, restock=20	70	Pass
TC2	sell()	quantity=70, sell=15	55	Pass
TC3	sell()	quantity=20, sell=25	ValueError	Pass
TC4	needs_reorder()	quantity=8, reorder=10	True	Pass
TC5	needs_reorder()	quantity=25, reorder=10	False	Pass
TC6	restock()+sell()	30, +10, -15	25	Pass
TC7	restock()+sell()	20, +5, -10	15	Pass
TC8	restock()+sell()	15, +20, -25	10	Pass
TC9	restock()+sell()	40, +0, -20	20	Pass

Reflection Paragraph

Through these three pytest homework tasks, I learned more about unit testing in Python and how important it is for checking different types of scenarios in code. I practiced writing tests for normal cases, boundary cases, and exceptional cases, which helped me understand program behavior better. I had some confusion about how parameterized testing actually works and how pytest treats each parameter set as a separate test case. After doing these tasks, that concept is much clearer to me now. I also became more comfortable using fixtures for setup and organizing test files. In the future, I would like to improve by writing more tricky edge-case tests.